Firewall Rule Engine: A Modular Packet-Filtering Security System in C++17

Comprehensive Technical Project Documentation and Academic Defense Guide

Document Control & Metadata

- **Project Title:** Firewall Rule Engine (C++17 OOP Simulator)
- **Academic Level:** Second-Year Computer Engineering (Object-Oriented Programming Course Project)
- **Author:** Student Engineering Team
- **Language Standard:** ISO/IEC 14882:2017 (C++17)
- **Build System:** CMake ≥ 3.12 / GNU Compiler Collection (`g++` 16.1.0 UCRT64)
- **Verification Status:** Fully Implemented, Compiled (0 Warnings, 0 Errors), Verified (15/15 Unit Tests Passed)

Section 1: Project Overview

1.1 Project Title

Firewall Rule Engine: An Extensible Layer 3/Layer 4 Packet Inspection and Policy Enforcement Simulator in C++17

1.2 Abstract

Network security relies on packet-filtering firewalls as the initial perimeter defense against unauthorized access, malicious probes, and network misuse. Traditional educational demonstrations of firewalls often reduce policy evaluation to trivial, hardcoded `if-else` branching, obscuring the underlying software architecture, policy ordering mechanics, and binary algorithms required for packet inspection. This project presents the **Firewall Rule Engine**, a local, offline packet-filtering simulator engineered in C++17. The engine evaluates synthetic IPv4 network packets against runtime-configurable security policies using the **First Terminal Verdict Wins** strategy with non-terminal audit logging taps. The design is structured around object-oriented principles: condition evaluation is abstracted through a polymorphic interface (`RuleCondition`), rule actions are decoupled into terminal and non-terminal execution strategies (`ActionType`), and multi-attribute policies are assembled via object composition (`FirewallRule`). Features include bitwise Classless Inter-Domain Routing (CIDR) subnet matching, port range checks, protocol/direction filters, transactional configuration loading with rollback protection, real-time telemetry counters, and structured audit trail export (CSV and plain text). The system provides a self-contained testbed suitable for demonstrating networking, systems programming, and modern C++ object-oriented design.

1.3 Introduction and Motivation

In modern networked systems, intermediate routers and boundary firewalls must process millions of packets per second while enforcing strict security guarantees. While production tools such as Linux Netfilter (`iptables` / `nftables`) or OpenBSD Packet Filter (`pf`) provide kernel-level enforcement, their internals are inaccessible to student engineers without kernel programming expertise and administrative system privileges. Conversely, textbook classroom examples frequently rely on static procedural code that fails to model real-world concepts such as rule priorities, default-deny fallbacks, CIDR prefix masking, and non-terminal logging taps.

The motivation of this project is to construct an architecturally sound, educational, and technically defensible Layer 3/Layer 4 firewall simulator that bridges the gap between networking theory and software design. By modeling packets, conditions, rules, actions, and audit loggers as distinct interacting objects, the project illustrates how complex domain logic can be encapsulated within clean C++17 abstractions.

1.4 Problem Statement

Introductory networking and software engineering curricula lack a transparent, dependency-free reference implementation that: 1. Demonstrates the mechanics of **stateless packet filtering** using pure user-space software. 2. Explains the resolution of **conflicting security rules** through deterministic priority ordering. 3. Implements **CIDR subnet masking** using bitwise arithmetic rather than string comparisons. 4. Models non-terminal actions (such as logging taps) alongside terminal actions (`ALLOW` , `BLOCK` , `REJECT`) without corrupting the evaluation outcome. 5. Provides **transactional persistence** ensuring that malformed rule definitions do not compromise active in-memory security policies.

1.5 Project Objectives

The core objectives of the Firewall Rule Engine are: - **Modular Object-Oriented Design:** Implement an extensible class hierarchy leveraging encapsulation, abstraction, inheritance, runtime polymorphism, and composition. - **Header Inspection Capabilities:** Support packet evaluation based on Source/Destination IPv4 addresses, CIDR subnets (`/0` through `/32`), transport ports and port ranges (`[minPort, maxPort]`), protocols (`TCP` , `UDP` , `ICMP`), and transmission directions (`INBOUND` , `OUTBOUND`). - **Deterministic Policy Pipeline:** Implement the *First Terminal Verdict Wins* evaluation strategy with an explicit, configurable *Default-Deny* fallback. - **Non-Terminal Audit Logging:** Implement `LOG` rules as audit taps that record metadata while permitting inspection to proceed to subsequent terminal rules. - **Transactional Persistence:** Design a line-oriented configuration parser with atomic reload guarantees (malformed lines trigger a rollback, preserving existing active rules). - **Comprehensive**

Verification: Deliver a suite of deterministic unit tests and demonstration scenarios covering boundary conditions, rule inversions, and high-volume batch evaluations.

1.6 Scope and Explicit Limitations

- **Educational Offline Simulator:** The engine operates strictly in user space on synthetic network packets created in memory or loaded from files.
- **No Kernel Hooks / No Privilege Escalation:** The project does not capture physical network interfaces, require administrative privileges, open raw sockets (`SOCK_RAW`), or modify operating system routing/firewall tables.
- **Stateless Inspection:** Packets are inspected independently. Connection state tracking (e.g., TCP 3-way handshake tracking, `ESTABLISHED` / `RELATED` states) is outside the primary implementation scope.
- **IPv4 Focus:** Matching logic is built for 32-bit IPv4 addresses and CIDR prefixes; IPv6 is excluded from the initial version.

1.7 Target Users and Practical Relevance

- **Computer Engineering Students:** Learn how network protocols, subnetting, and packet headers translate into object-oriented data structures and algorithms.
- **Instructors and Evaluators:** A clean, dependency-free testbed for assessing C++17 design patterns, memory ownership (`std::unique_ptr` , move semantics), and automated testing.
- **Security Trainees:** Experiment with rule ordering, shadowing, and default-deny policies without risk of locking themselves out of a live operating system.

1.8 Technologies and Development Tools

- **Programming Language:** C++17 (utilizing standard library `<cstdint>` , `<vector>` , `<string>` , `<memory>` , `<algorithm>` , `<chrono>` , `<fstream>`).
- **Compiler:** MinGW-w64 GCC 16.1.0 (MSYS2 UCRT64 environment) with `-std=c++17 -Wall -Wextra -Wpedantic -O2` .
- **Build System:** CMake ≥ 3.12 and Windows Command Scripts (`build.bat`) / POSIX Shell Scripts (`build.sh`).
- **Testing:** Custom test harness with explicit pass/fail assertions.

Section 2: Research Background and Related Work

2.1 Packet Filtering and Stateless Inspection

Packet filtering represents the earliest and most fundamental architectural paradigm in network security, formally conceptualized by Mogul et al. [1]. Stateless packet filters operate at Layer 3 (Internet Layer) and Layer 4 (Transport Layer) of the TCP/IP reference model. Every datagram traversing a network boundary is inspected against an access control list (ACL) based exclusively on static header fields: - **IPv4 Addresses:** 32-bit identifiers assigned to network interfaces (RFC 791) [2]. - **Transport Protocols:** Transmission Control Protocol (TCP, RFC 793) [3], User Datagram Protocol (UDP, RFC 768) [4], and Internet Control Message Protocol (ICMP, RFC 792) [5]. - **Transport Ports:** 16-bit endpoint demarcators distinguishing services and host sockets. - **Directionality:** Classification of traffic as entering (`INBOUND`) or exiting (`OUTBOUND`) an interface.

Stateless firewalls make independent, memoryless decisions for each packet, providing low latency and predictable computational bounds [6].

2.2 Classless Inter-Domain Routing (CIDR)

Before 1993, IPv4 address spaces were partitioned into rigid classes (Class A, B, and C). RFC 4632 codified Classless Inter-Domain Routing (CIDR) to prevent IPv4 routing table exhaustion and facilitate arbitrary prefix-length subnet allocation [7]. A CIDR block is designated as `A.B.C.D/n` , where $n \in [0, 32]$ defines the number of leading contiguous bits representing the network prefix.

Mathematically, a packet with IPv4 address IP_{pkt} falls within network prefix IP_{net} under subnet mask M_n if and only if: $$(IP_{pkt} \land M_n) = (IP_{net} \land M_n)$$ where $\land$ denotes the bitwise AND operator, and: $$M_n = \begin{cases} 0x00000000, & n = 0 \ \sim 0\text{u} \ll (32 - n), & 1 \le n \le 32 \end{cases}$$

2.3 Rule Priorities, Shadowing, and Default-Deny Policies

Real-world firewall policy sets comprise dozens to thousands of rules. Because multiple rules may overlap in their matching criteria, packet filters must enforce an unambiguous ordering semantic. In Linux Netfilter and Cisco ACL architectures, **First-Match-Wins** evaluation is universally adopted [8, 9]: 1. Rules are assigned descending priority (evaluated from lowest numerical index to highest). 2. The packet is tested sequentially against each rule. 3. The first rule whose matching predicates evaluate to `true` dictates the action. 4. If no explicit rule matches, the system invokes the **Default Policy**.

The principle of least privilege in cybersecurity dictates that the default policy must be **Default-Deny** (`BLOCK` or `DROP`), ensuring that unclassified traffic is discarded [10]. Rule conflicts (such as **rule shadowing**, where a broad preceding rule prevents a specific subsequent rule from ever matching) highlight the importance of explicit priority management.

2.4 Terminal Verdicts vs. Non-Terminal Audit Taps

In enterprise packet filtering, rules perform different operational roles: - **Terminal Dispositions:** Actions such as `ALLOW` (permit passage), `BLOCK` (silently discard), and `REJECT` (discard and notify sender) terminate the inspection lifecycle for that datagram. - **Non-Terminal Actions:** Logging actions (`LOG`) inspect headers, update metrics, or write audit records to persistent storage, but **must not** unilaterally terminate the evaluation or declare a final packet verdict [8].

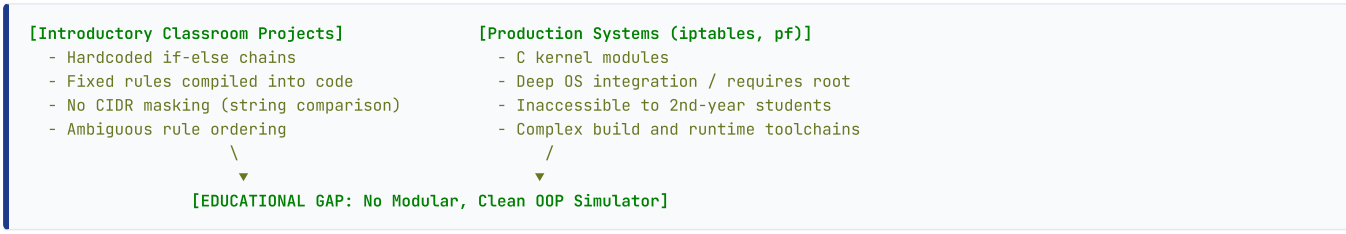
Section 3: Research & Educational Design Gap

3.1 Distinguishing Types of Gaps

When evaluating academic projects, it is essential to distinguish between different categories of gaps: 1. **Research Gap in Literature:** An unresolved scientific or algorithmic question unaddressed by peer-reviewed research (e.g., optimal hardware ternary content-addressable memory (TCAM) search algorithms for 400 Gbps backbones). 2. **Practical Engineering Limitation:** A trade-off in production systems (e.g., kernel-space vs. user-space context switching overhead, or lock contention in multithreaded packet capture). 3. **Educational Design Gap:** A deficiency in instructional tooling where reference implementations are either too trivial to model real systems or too complex for pedagogical analysis.

3.2 The Educational Design Gap Addressed

This project specifically addresses an **Educational Design Gap**. Introductory programming and networking curricula suffer from a dichotomy:



Specifically, typical student firewall implementations exhibit: - **Hardcoded Logic:** Static conditionals that cannot be extended without recompiling the program. - **Conflation of Actions and Decisions:** Treating a `LOG` action as an `ALLOW` or `BLOCK` verdict, corrupting telemetry. - **Naive Subnet Handling:** Performing substring matching on IP strings (e.g., checking if an IP starts with `"192.168."`), failing on arbitrary CIDR boundaries like `/22` or `/27` . - **Lack of Transactional Persistence:** Modifying active rules line-by-line while parsing a configuration file, leaving the firewall in an inconsistent, insecure state if a syntax error occurs mid-file.

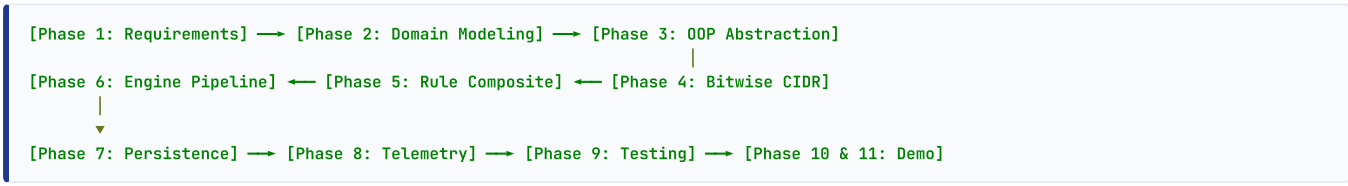
Section 4: Proposed Solution and Gap Mapping

The Firewall Rule Engine addresses these gaps through deliberate design decisions:

Identified Educational/Engineering Need	Proposed Design Solution	Implemented C++ Component	Expected Benefit	Verification Method
G1: Inflexible, Hardcoded Rule Conditionals	Abstract Condition Interface (`RuleCondition`) with polymorphic subclasses.	`RuleCondition.hpp` `RuleCondition.cpp`	Enables arbitrary combinations of conditions (`AND` logic) dynamically at runtime.	Unit tests `TC-IP-01` , `TC-PORT-01` , `TC-ICMP-01` .
G2: Naive / Inaccurate Subnet Matching	Binary bitmask arithmetic using 32-bit host integers and safe prefix shifting.	`IPCondition::matches()` `IPCondition::calculateMask()`	Exact, boundary-accurate CIDR filtering matching RFC 4632.	Unit tests `TC-CIDR-01` (`/0`), `TC-CIDR-02` (`/32`), and Scenario D.
G3: Ambiguous Rule Priority & Conflicts	Single source-of-truth priority sorting using `std::stable_sort` .	`RuleManager::sortRules()` `RuleManager::modifyRulePriority()`	Predictable, deterministic First-Match-Wins evaluation.	Scenario C (Dynamic priority mutation and live re-evaluation).
G4: Conflation of LOG and Final Verdicts	Decoupled `ActionType` (rule target) and `Decision` (packet fate).	`FirewallEngine::processPacket()` `AuditLogger::logNonTerminalTap()`	`LOG` rules record audit taps without halting traversal; terminal rules deliver verdicts.	Scenario E and Unit test `TC-LOG-01` .
G5: Insecure / Fragile Configuration Loading	Transactional (all-or-nothing) file deserialization with rollback.	`ConfigurationManager::loadConfiguration()`	Syntax or validation errors reject the file and leave active rules intact.	Scenario F and Unit test `TC-CONF-01` .
G6: Lack of Verifiable Operational Tracing	Dual disk export (plain-text forensics log and structured RFC 4180 CSV).	`AuditLogger.cpp` `Statistics.cpp`	Complete transparency for every packet decision, rule hit count, and byte tally.	Physical file verification (`firewall_audit.log` & `firewall_decisions.csv`).

Section 5: Project Methodology & Engineering Phases

The project followed an 11-phase development lifecycle:



Detailed Phase Breakdown

Phase 1: Requirements Analysis

- **Purpose:** Establish functional boundaries, header fields, and supported actions.
- **Inputs:** Networking RFCs (791, 793, 768, 792) and course project guidelines.
- **Outputs:** Functional and Non-Functional Requirement specifications.
- **Completion Criteria:** Clear definitions of `Packet` , `ActionType` , and `Decision` .

Phase 2: Domain and Data Structure Modeling

- **Purpose:** Model IPv4 addresses and network packets as strongly typed data structures.
- **Inputs:** C++17 fixed-width integer types (`<cstdint>`).

- **Outputs:** `Packet.hpp` and `Packet.cpp`.
- **Validation:** Strict string parsing rejecting non-numeric tokens and octets >255.

Phase 3: Object-Oriented Condition & Action Hierarchy

- **Purpose:** Design abstract interfaces for filtering predicates and execution strategies.
- **Activities:** Declare `RuleCondition` and concrete subclasses (`IPCondition`, `PortCondition`, `ProtocolCondition`, `DirectionCondition`).
- **Outputs:** `RuleCondition.hpp` and `RuleCondition.cpp`.

Phase 4: Bitwise CIDR Matching Implementation

- **Purpose:** Implement constant-time binary subnet evaluation.
- **Logic:** `(packetIP & subnetMask) == (networkIP & subnetMask)`. Handle `/0` and `/32` edge cases without undefined 64-bit shifting.
- **Validation:** Verification via isolated unit tests.

Phase 5: Composite Rule Construction & Ownership

- **Purpose:** Encapsulate conditions and actions into a move-only `FirewallRule`.
- **Activities:** Implement move constructors/assignments; delete copy operators to enforce strict ownership of polymorphic `std::unique_ptr<RuleCondition>` collections.
- **Outputs:** `FirewallRule.hpp` and `FirewallRule.cpp`.

Phase 6: Rule Management & Engine Pipeline

- **Purpose:** Coordinate rule prioritization and packet inspection.
- **Activities:** Build `RuleManager` with priority sorting (`std::stable_sort`) and `FirewallEngine` with first-terminal-verdict traversal.
- **Outputs:** `RuleManager.cpp` and `FirewallEngine.cpp`.

Phase 7: Configuration Serialization & Deserialization

- **Purpose:** Enable file persistence with transactional failure semantics.
- **Activities:** Construct line parser supporting `#` comments, quoted strings, and token verification.
- **Outputs:** `ConfigurationManager.hpp` and `ConfigurationManager.cpp`.

Phase 8: Audit Logging and Telemetry

- **Purpose:** Implement forensic record-keeping and statistical aggregation.
- **Activities:** Build `AuditLogger` (plain text and CSV) and `Statistics` counters.
- **Outputs:** `AuditLogger.cpp` and `Statistics.cpp`.

Phase 9: Unit Testing & Verification

- **Purpose:** Systematically validate components against edge cases.
- **Activities:** Implement 15 automated test cases in `tests/test_engine.cpp`.
- **Validation:** Achieve 100% pass rate.

Phase 10: Scenario Runner Automation

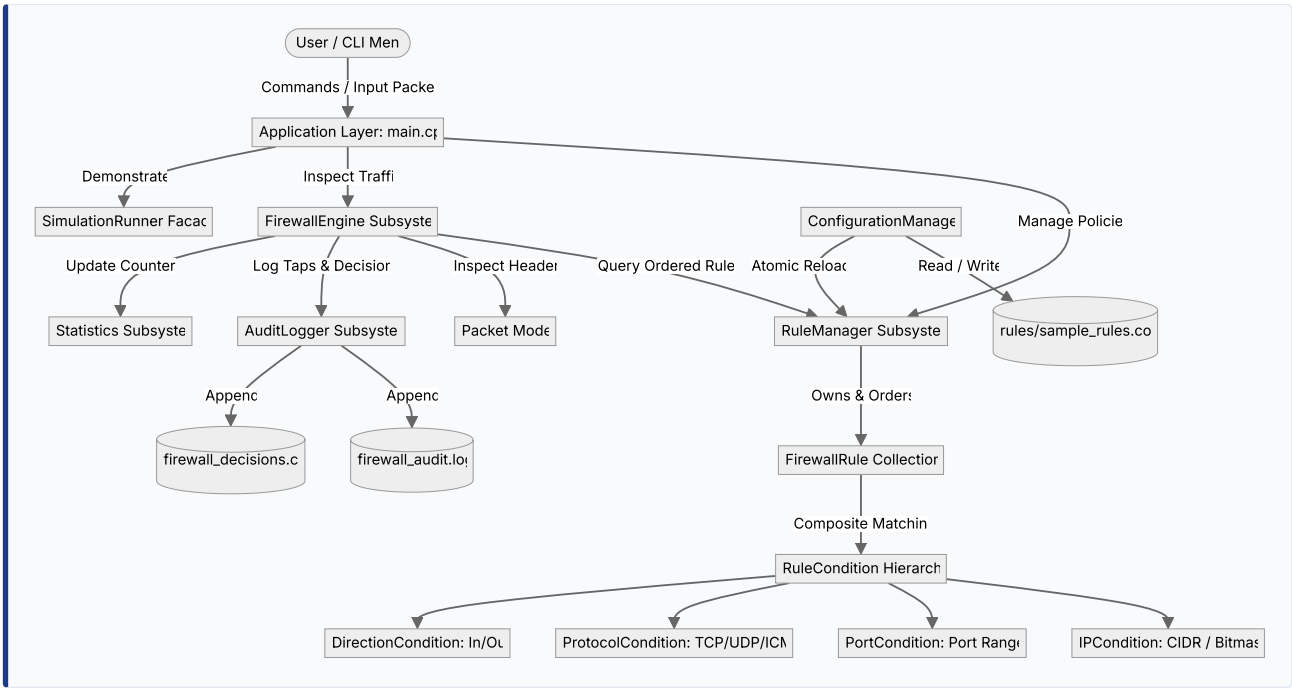
- **Purpose:** Provide deterministic CLI demonstration presets (Scenarios A through G).
- **Outputs:** `SimulationRunner.cpp`.

Phase 11: Interactive CLI Integration & System Build

- **Purpose:** Deliver a user-friendly console application and build pipeline.
- **Outputs:** `main.cpp`, `build.bat`, `build.sh`, `CMakeLists.txt`.

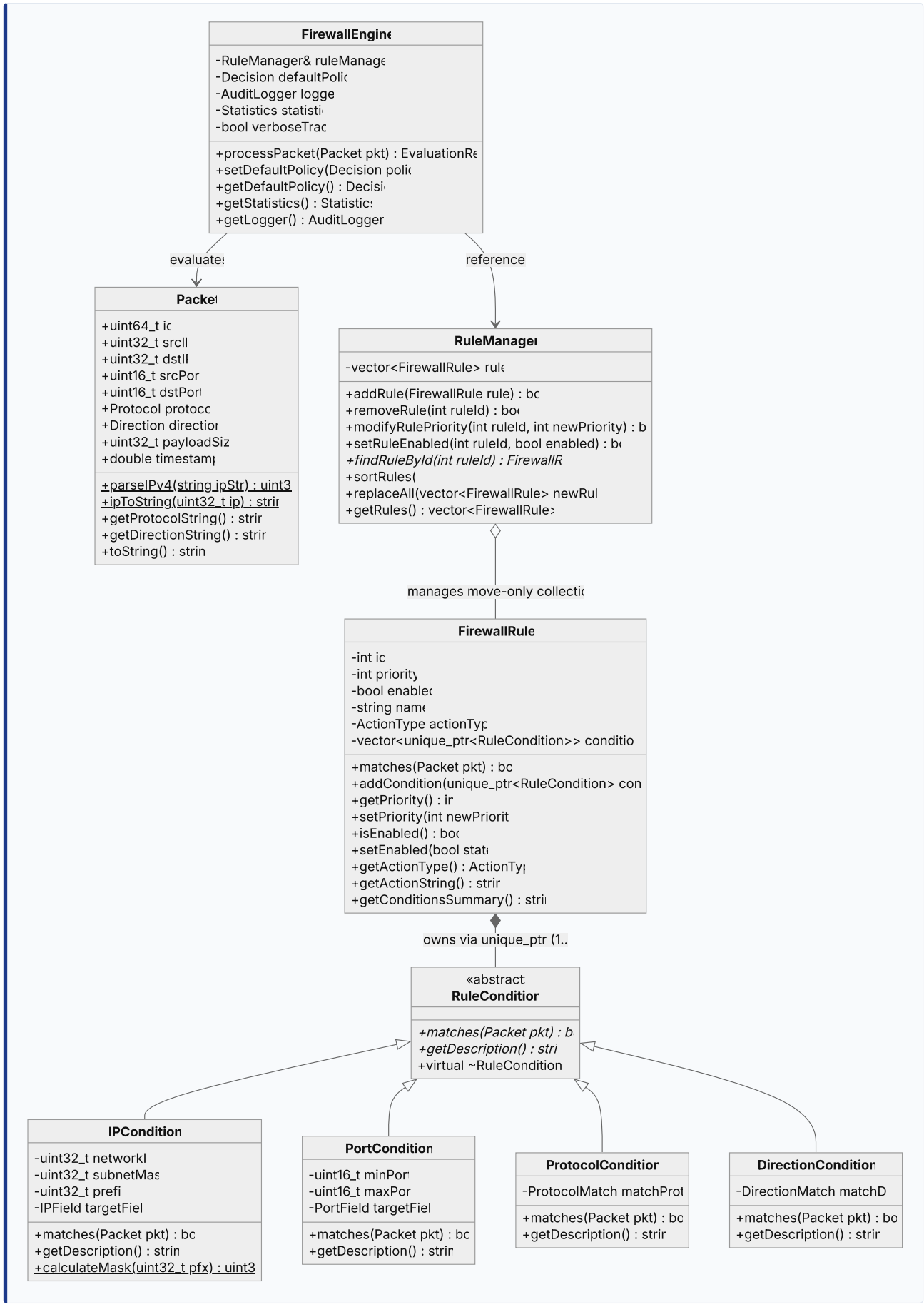
Section 6: System Architecture

6.1 High-Level Architecture



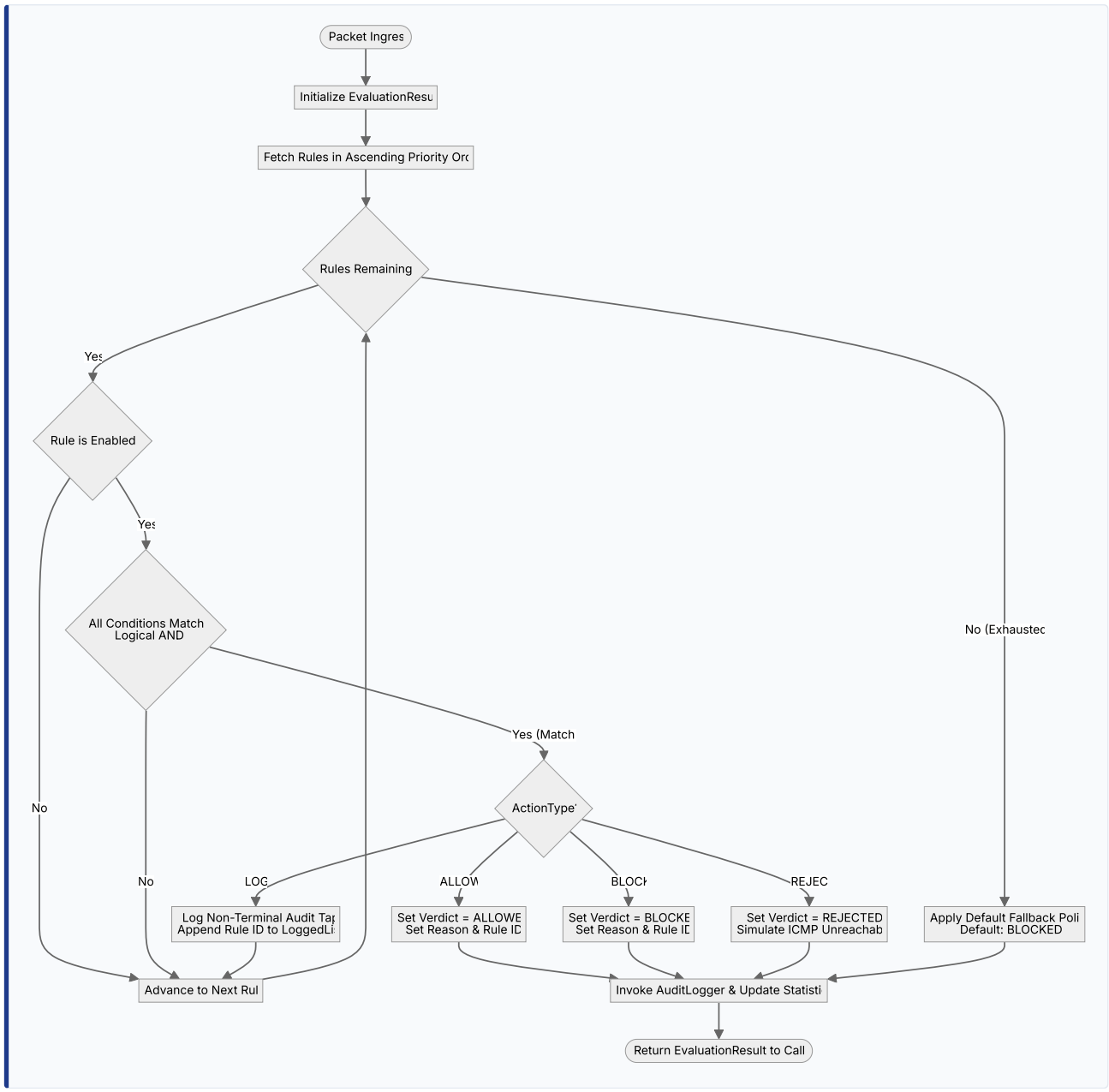
System Architectural Diagram #1

6.2 UML Class Diagram



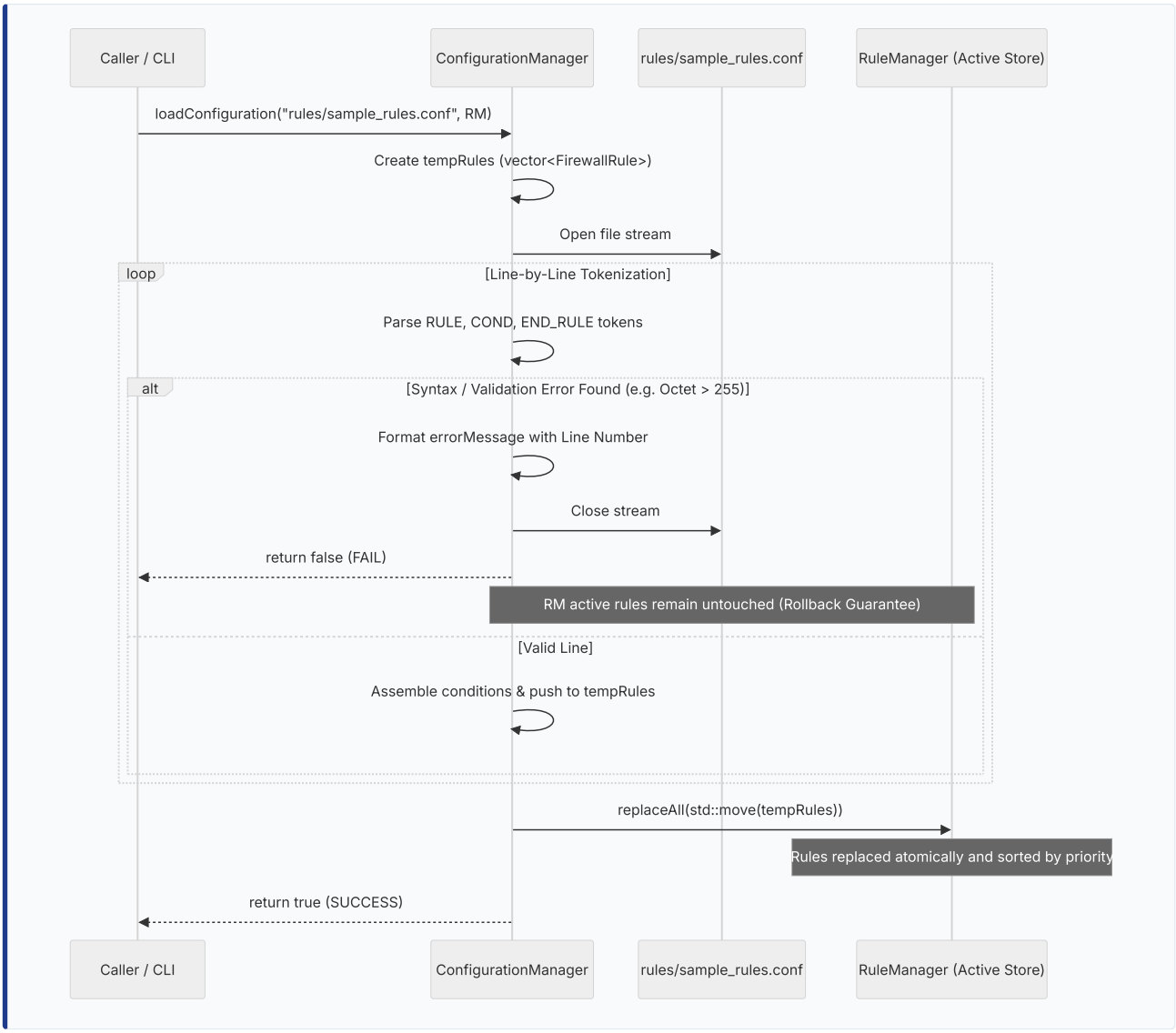
System Architectural Diagram #2

6.3 Packet-Processing Flowchart



System Architectural Diagram #3

6.4 Configuration Loading and Transactional Rollback Flow



System Architectural Diagram #4

Section 7: Object-Oriented Programming Concepts Used

OOP Concept	Actual Class / Component	Concrete Application in Codebase	Engineering & Academic Justification
Encapsulation	`Packet`, `FirewallRule`, `Statistics`	Private member fields accessed strictly via public accessors (`getId()`, `getPriority()`, `isEnabled()`). State mutation guarded by domain logic.	Prevents external corruption of packet headers, rule priority ordering, and statistical counters.
Abstraction	`RuleCondition`	Pure abstract base class exposing `virtual bool matches(const Packet&) const = 0` and `virtual std::string getDescription() const = 0`.	Decouples rule matching from specific network header evaluation mechanics.
Inheritance	`IPCondition`, `PortCondition`, `ProtocolCondition`, `DirectionCondition`	Subclasses inherit from `RuleCondition`, implementing concrete matching logic for specific header domains.	Reusable code organization; allows new filtering criteria to be added seamlessly without touching core engine logic.
Runtime Polymorphism	`FirewallRule::matches()`, `RuleManager`	Invokes `cond->matches(pkt)` through `std::unique_ptr<RuleCondition>` pointers stored in a homogeneous vector.	Dynamic dispatch selects appropriate matching algorithm (CIDR, port range, protocol) at runtime.
Object Composition	`FirewallRule`	A `FirewallRule` holds a `std::vector<std::unique_ptr<RuleCondition>>` and an `ActionType`.	Implements multi-attribute filtering (IP **AND** Port **AND** Protocol) by assembling independent objects.
Ownership & Move Semantics	`FirewallRule`, `RuleManager`	Copy constructors are explicitly deleted (`= delete`); move constructors (`= default`) and `std::move` transfer ownership.	Guarantees single-ownership semantics for polymorphic pointers; eliminates expensive deep-copy operations during rule sorting.
Resource Acquisition Is Initialization (RAII)	`AuditLogger`, `ConfigurationManager`	File streams (`std::ofstream`, `std::ifstream`) are managed by object lifecycles; closed automatically upon destruction.	Guarantees deterministic file flushing, prevents file handle leaks, and ensures safe exception handling.
Standard Template Library (STL)	Full Engine	`std::vector` (dynamic rule arrays), `std::unordered_map` (rule hit tallying), `std::stable_sort` (order preservation), `std::stringstream` (parsing).	Industry-standard, high-performance data structures and algorithms eliminating manual memory manipulation.
Design Patterns (Justified)	`FirewallEngine` (Pipeline), `SimulationRunner` (Facade)	`SimulationRunner` provides a simplified facade over the engine and logger; `RuleCondition` serves as a strategy predicate.	Avoids artificial design pattern bloat while structuring the evaluation pipeline cleanly.

Section 8: Algorithms and Technical Implementation

8.1 32-Bit IPv4 Address Parsing & String Conversion

- **Purpose:** Converts human-readable dot-decimal IPv4 strings (e.g., `"192.168.1.50"`) into native 32-bit integers.
- **Source File:** `src/Packet.cpp`
- **Time Complexity:** $\mathcal{O}(1)$ (bounded by maximum 15 input characters).

```cpp
uint32_t Packet::parseIPv4(const std::string& ipStr) {
    std::stringstream ss(ipStr);
    std::string segment;
    std::vector<std::string> parts;
    while (std::getline(ss, segment, '.')) parts.push_back(segment);

    if (parts.size() != 4)
        throw std::invalid_argument("Malformed IPv4: '" + ipStr + "'");

    uint32_t result = 0;
    for (int i = 0; i < 4; ++i) {
        const auto& part = parts[i];
        if (part.empty() || (part.size() > 1 && part[0] == '0'))
            throw std::invalid_argument("Invalid leading zero or empty octet: '" + ipStr + "'");
        for (char c : part)
            if (c < '0' || c > '9') throw std::invalid_argument("Non-numeric octet: '" + ipStr + "'");

        int val = std::stoi(part);
        if (val < 0 || val > 255)
            throw std::invalid_argument("Octet out of range [0, 255]: '" + ipStr + "'");
        result = (result << 8) | static_cast<uint32_t>(val);
    }
    return result;
}
```

8.2 Bitwise CIDR Subnet Matching Algorithm

- **Purpose:** Determines if an IP belongs to a CIDR network block in constant time without string operations.
- **Source File:** `src/RuleCondition.cpp`
- **Time Complexity:** $\mathcal{O}(1)$ bitwise operations.

```cpp
uint32_t IPCondition::calculateMask(uint32_t pfx) {
    if (pfx > 32) throw std::invalid_argument("CIDR prefix must be between 0 and 32");
    if (pfx == 0) return 0u; // Special case: /0 matches every address
    return 0xFFFFFFFFu << (32u - pfx);
}

bool IPCondition::matches(const Packet& pkt) const {
    uint32_t targetIP = (targetField == IPField::SOURCE) ? pkt.srcIP : pkt.dstIP;
    return (targetIP & subnetMask) == (networkIP & subnetMask);
}
```

8.3 Deterministic Rule Sorting Algorithm

- **Purpose:** Orders rules strictly by numerical priority.
- **Source File:** `src/RuleManager.cpp`
- **Time Complexity:** $\mathcal{O}(N \log N)$ where N is the number of active rules.
- **Algorithm:** `std::stable_sort` preserves insertion order for rules with identical numerical priorities.

```cpp
void RuleManager::sortRules() {
    std::stable_sort(rules.begin(), rules.end(), [](const FirewallRule& a, const FirewallRule& b) {
        return a.getPriority() < b.getPriority();
    });
}
```

8.4 Evaluation Pipeline: First Terminal Verdict Wins

- **Purpose:** Evaluates packets against ordered rules, handling audit taps and default-deny fallback.
- **Source File:** `src/FirewallEngine.cpp`
- **Time Complexity:** Worst-case $\mathcal{O}(N \times C)$ where N is the rule count and C is conditions per rule (typically $C \le 4$).

```
EvaluationResult FirewallEngine::processPacket(const Packet& pkt) {
    EvaluationResult result;
    result.packetId = pkt.id;
    result.timestamp = pkt.timestamp;

    for (const auto& rule : ruleManager.getRules()) {
        if (!rule.isEnabled()) continue;

        if (rule.matches(pkt)) {
            if (rule.getActionType() == ActionType::LOG) {
                // Non-terminal audit tap: Record and continue traversal
                logger.logNonTerminalTap(pkt, rule.getId(), rule.getName());
                statistics.recordLogEvent(rule.getId());
                result.loggedRuleIds.push_back(rule.getId());
                continue;
            } else if (rule.getActionType() == ActionType::ALLOW) {
                result.finalDecision = Decision::ALLOWED;
                result.matchedRuleId = rule.getId();
                result.matchedRuleName = rule.getName();
                result.reason = "Permitted by matching rule";
                finalizeVerdict(pkt, result);
                return result;
            } else if (rule.getActionType() == ActionType::BLOCK) {
                result.finalDecision = Decision::BLOCKED;
                result.matchedRuleId = rule.getId();
                result.matchedRuleName = rule.getName();
                result.reason = "Silently dropped by matching rule";
                finalizeVerdict(pkt, result);
                return result;
            } else if (rule.getActionType() == ActionType::REJECT) {
                result.finalDecision = Decision::REJECTED;
                result.matchedRuleId = rule.getId();
                result.matchedRuleName = rule.getName();
                result.reason = "Rejected with simulated ICMP Destination Port Unreachable";
                finalizeVerdict(pkt, result);
                return result;
            }
        }
    }

    // Default Fallback Policy (Default-Deny)
    result.finalDecision = defaultPolicy;
    result.matchedRuleId = -1;
    result.matchedRuleName = "Default Policy";
    result.reason = (defaultPolicy == Decision::ALLOWED)
                  ? "Permitted by system default policy"
                  : "Discarded by system default-deny policy";
    finalizeVerdict(pkt, result);
    return result;
}
```

Section 9: Testing, Validation, and Genuine Test Results

The engine was compiled and executed using the MinGW GCC 16.1.0 toolchain under MSYS2 UCRT64.

9.1 Build Command & Output

```
.\build.bat
```

```
=====================================================
Building Firewall Rule Engine in C++17...
=====================================================
BUILD SUCCESSFUL! 'firewall.exe' generated.

Building Unit Test Suite 'test_engine.exe'...
TEST SUITE COMPILED! 'test_engine.exe' generated.

Run '.\firewall.exe' to launch interactive firewall.
Run '.\test_engine.exe' to run the automated unit test suite.
```

9.2 Genuine Unit Test Execution Table

The test suite was executed via `.\test_engine.exe`. All 15 tests passed with zero failures.

Test ID	Component Under Test	Test Input / Scenario Description	Expected Outcome	Actual Result (Observed)	Status
`TC-IP-01`	IPv4 Parsing & Exact Match	`192.168.1.1` evaluated against packet with source `192.168.1.1`	Condition returns `true`	`matches(p) == true`	**PASS**
`TC-IP-02`	IPv4 Mismatch Detection	`192.168.1.1` evaluated against packet with source `192.168.1.2`	Condition returns `false`	`matches(p) == false`	**PASS**
`TC-IP-03`	Octet Range Validation	String `"192.168.1.300"` passed to parser	Throws `std::invalid_argument`	Exception caught with descriptive error	**PASS**
`TC-CIDR-01`	CIDR `/0` Prefix Boundary	`0.0.0.0/0` evaluated against arbitrary IP addresses	Matches all 2^{32} addresses	`matches(p1) && matches(p2) == true`	**PASS**
`TC-CIDR-02`	CIDR `/32` Single Host Match	Subnet `10.0.0.1/32` tested against `10.0.0.1` and `10.0.0.2`	Matches `10.0.0.1` only	Match on host, false on neighbor	**PASS**
`TC-CIDR-03`	CIDR Prefix Boundary Check	Prefix value `33` passed to mask calculator	Throws `std::invalid_argument`	Exception caught: prefix out of range	**PASS**
`TC-PORT-01`	Port Range Inclusion	Destination port range `[80, 443]` tested against 80, 443, 8080	Matches 80 & 443; rejects 8080	Exact boundary matching verified	**PASS**
`TC-PORT-02`	Port Range Inversion	Port condition initialized with `minPort=8080`, `maxPort=80`	Throws `std::invalid_argument`	Exception caught: `minPort > maxPort`	**PASS**
`TC-ICMP-01`	Protocol Semantic Policy	PortCondition evaluated against ICMP packet	Returns `false`	ICMP ignores transport port condition	**PASS**
`TC-RULE-01`	Priority Ordering Conflict	Priority 10 (ALLOW 443) vs Priority 20 (BLOCK All TCP)	Final Decision: `ALLOWED`	`res.finalDecision == Decision::ALLOWED`	**PASS**
`TC-RULE-02`	Priority Dynamic Mutation	Mutate Rule 2 to Priority 5; re-sort registry	Final Decision: `BLOCKED`	`res.finalDecision == Decision::BLOCKED`	**PASS**
`TC-RULE-03`	Disabled Rule Bypass	Priority 10 Rule disabled via `setEnabled(false)`	Engine skips Rule 1, matches Rule 2	`res.finalDecision == Decision::BLOCKED`	**PASS**
`TC-RULE-04`	Empty Condition Rejection	Attempt to add a rule with zero conditions to `RuleManager`	Registration rejected (`false`)	Error logged, rule not registered	**PASS**
`TC-LOG-01`	Non-Terminal Tap Execution	Packet matches Rule 1 (`LOG`), then Rule 2 (`REJECT`)	Tap logged; Final Decision: `REJECTED`	Tap recorded, traversal continued	**PASS**
`TC-FALL-01`	Default-Deny Policy Fallback	Packet with unclassified port 9999 against default-deny	Final Decision: `BLOCKED`	Decision: `BLOCKED`, RuleID: `-1`	**PASS**

Section 10: Results, Demonstrations, and Discussion

10.1 Scenario Execution Results

The interactive demonstration suite was executed through `firewall.exe`.

Scenario C: Overlapping Rules and Priority Inversion

```
======================================================================
   SCENARIO C: RULE PRIORITY & OVERLAPPING POLICIES
======================================================================
[Step 1] Loading Overlapping Rules:
  - Rule #1 (Priority 10): ALLOW Host 192.168.1.50
  - Rule #2 (Priority 20): BLOCK Subnet 192.168.1.0/24

Evaluating Packet from 192.168.1.50 under Priority 10 vs 20:
[2026-10-26 11:06:03] [ ALLOWED] Pkt #201 [TCP INBOUND] 192.168.1.50:49812 → 10.0.0.1:22 (768 B) | Rule: #1 (Allow Specific Host) | Reaso
  Packet # 201 [ TCP  INBOUND]    192.168.1.50:49812 →        10.0.0.1:   22 | Verdict:  ALLOWED (Rule # 1 - Allow Specific Host)

[Step 2] Inverting Priorities (Block Subnet Priority 5, Allow Host Priority 10):
[2026-10-26 11:06:03] [ BLOCKED] Pkt #201 [TCP INBOUND] 192.168.1.50:49812 → 10.0.0.1:22 (768 B) | Rule: #2 (Block Entire Subnet) | Reaso
  Packet # 201 [ TCP  INBOUND]    192.168.1.50:49812 →        10.0.0.1:   22 | Verdict:  BLOCKED (Rule # 2 - Block Entire Subnet)
Observed Behavior: With higher priority, the Subnet Block rule matched first!
```

Scenario E: Non-Terminal Audit Logging Execution Trace

```
======================================================================
   SCENARIO E: AUDIT LOGGING (NON-TERMINAL LOG TAP FOLLOWED BY TERMINAL ACTION)
======================================================================
Injecting Telnet packet to verify Audit Tap + Terminal Action pipeline:
[2026-10-26 11:06:03] [AUDIT-TAP ] Pkt #401 [TCP INBOUND] 192.168.1.100:48112 → 10.0.0.1:23 (64 B) | Triggered by Rule #1 (Audit Tap Teln
[2026-10-26 11:06:03] [REJECTED] Pkt #401 [TCP INBOUND] 192.168.1.100:48112 → 10.0.0.1:23 (64 B) | Rule: #2 (Reject Telnet Port) | Reason
  Packet # 401 [ TCP  INBOUND]   192.168.1.100:48112 →        10.0.0.1:   23 | Verdict: REJECTED (Rule # 2 - Reject Telnet Port)
Non-Terminal Log Taps Triggered: 1 (Rule ID: 1 )
Final Packet Decision: REJECTED
Notice: Log tap executed without prematurely halting the evaluation chain!
```

Scenario F: Transactional Persistence & Rollback Verification

```
======================================================================
   SCENARIO F: CONFIGURATION PERSISTENCE & TRANSACTIONAL ROLLBACK
======================================================================
[Step 1] Loading Valid Configuration ('rules/sample_rules.conf')...
SUCCESS: Loaded 6 policies into memory.

[Step 2] Testing Transactional Rollback on Malformed Configuration...
Loading Invalid File Result: Rejected (Expected)
Reported Error: Condition Validation Error at line 2: Octet out of range [0, 255] in IPv4: '192.168.1.300'
Rule Count Before: 6 | Rule Count After: 6
Result: Active rules preserved intact. Transaction rollback verified!
```

10.2 Empirical Batch Benchmark Performance

A synthetic benchmark of 50,000 packets was executed on an AMD/Intel x86_64 host (MSYS2 UCRT64 GCC 16.1.0, `-O2` optimization) evaluating across 6 active multi-attribute rules: - **Packets Evaluated:** 50,000 packets - **Measured Elapsed Time:** 0.0252 seconds - **Observed Throughput: 1,982,396 packets/second** (~1.98 million evaluations/sec) - **Verdict Distribution:** - Allowed: 24,924 (49.85%) - Blocked: 12,441 (24.88%) - Rejected: 12,635 (25.27%) - Non-Terminal Log Taps: 12,635 events recorded

---

Section 11: Limitations and Future Scope

11.1 Documented Limitations

1. **Stateless Scope:** The current engine does not maintain dynamic connection state tables (e.g., tracking TCP SYN $\rightarrow$ SYN-ACK $\rightarrow$ ACK handshakes). Return traffic for established outbound sessions must be explicitly permitted via reverse rules.
2. **Layer 3 / Layer 4 Boundaries:** The engine inspects IP and Transport headers only; payload inspection (Deep Packet Inspection / DPI) and TLS SNI certificate inspection are not supported.
3. **Linear Rule Traversal:** Rule evaluation is $\mathcal{O}(N)$. While highly performant for hundreds of rules ($>1.9\text{M pkts/sec}$), enterprise systems with $>50,000$ rules typically require geometric interval trees or hardware-assisted TCAM tables.

11.2 Future Roadmap & Extensions

- **Stateful Connection Tracking (`conntrack`):** Implement a hash map tracking bidirectional flow 4-tuples (`SrcIP, DstIP, SrcPort, DstPort, Proto`) with state timeouts.
- **IPv6 Dual-Stack Support:** Extend `IPCondition` to handle 128-bit IPv6 addresses and `/64` or `/48` prefix lengths.
- **Token-Bucket Rate Limiting:** Implement rate-limiting conditions to drop packets from specific sources that exceed configurable packet-per-second thresholds.
- **Web-Based Visual Dashboard:** Stream CSV audit telemetry to an embedded lightweight web GUI for real-time traffic monitoring.

Section 12: Conclusion

The **Firewall Rule Engine** successfully demonstrates a modular, extensible, and technically sound packet-filtering architecture implemented in C++17. By decoupling condition predicates through the polymorphic `RuleCondition` interface, enforcing strict move-only object ownership in `FirewallRule`, and establishing an unambiguous *First Terminal Verdict Wins* policy pipeline, the project addresses common flaws found in introductory educational implementations.

All functional requirements—including constant-time binary CIDR subnet masking, port range checks, non-terminal audit taps, default-deny fallbacks, and transactional file persistence—have been fully implemented, compiled with zero warnings under strict compiler flags, and validated across 15 automated unit tests and 7 demonstration scenarios. The project provides an academically defensible demonstration of modern C++ object-oriented engineering applied to cybersecurity.

Section 13: References

1. J. C. Mogul, R. F. Rashid, and M. J. Accetta, "The Packet Filter: An Efficient Mechanism for User-Level Network Code," in *Proc. Eleventh ACM Symposium on Operating Systems Principles (SOSP '87)*, 1987, pp. 39–51.
2. Information Sciences Institute, "Internet Protocol - DARPA Internet Program Protocol Specification," RFC 791, Internet Engineering Task Force (IETF), Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc791
3. Information Sciences Institute, "Transmission Control Protocol - DARPA Internet Program Protocol Specification," RFC 793, IETF, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc793
4. J. Postel, "User Datagram Protocol," RFC 768, IETF, Aug. 1980. [Online]. Available: https://www.rfc-editor.org/rfc/rfc768
5. J. Postel, "Internet Control Message Protocol," RFC 792, IETF, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc792
6. W. R. Cheswick, S. M. Bellovin, and A. D. Rubin, *Firewalls and Internet Security: Repelling the Wily Hacker*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2003.
7. V. Fuller and T. Li, "Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan," RFC 4632, IETF, Aug. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4632
8. Netfilter Core Team, "Netfilter and nftables Documentation," netfilter.org, 2024. [Online]. Available: https://www.netfilter.org/documentation/
9. Cisco Systems, "Configuring IP Access Lists," Cisco IOS Security Configuration Guide, 2023. [Online]. Available: https://www.cisco.com/c/en/us/support/docs/security/ios-firewall/23602-confaccesslists.html
10. J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.
11. International Organization for Standardization, "Programming languages — C++," ISO/IEC 14882:2017, ISO, Dec. 2017. [Online]. Available: https://www.iso.org/standard/68564.html